Professional Self-Assessment

Dave Mobley

Southern New Hampshire University

CS-499

Dr. Penmatsa

December 14, 2025

Professional Self-Assessment

Introduction

Completing the Computer Science program and developing this ePortfolio strengthened my ability to turn ideas into software that is **understandable, secure, testable, and deliverable**. Across courses and milestones, I learned to treat software as a socio-technical system: engineering choices are inseparable from stakeholder needs, communication clarity, operational realities, and security risks.

This self-assessment serves as the formal introduction to the ePortfolio. It summarizes the values and competencies I developed across the program and explains how the artifacts that follow fit together as a cohesive demonstration of my growth.

How the program shaped my professional goals and values

Throughout the program, I moved from focusing primarily on "getting the code to work" to focusing on **making systems reliable, maintainable, and defensible**:

- I learned to communicate design intent and trade-offs in a way that supports decision making.
- I learned to select data structures and algorithms based on workload patterns, not just theoretical complexity.
- I learned to treat databases as systems that require index design, query shaping, and analytics strategy.
- I learned to approach security as a mindset—anticipating how systems fail, how they are misused, and how to reduce risk early.

These values directly influence how I want to be positioned professionally: as a technically strong engineer with leadership capability, able to work across stakeholders and deliver production-oriented solutions.

Collaboration in a team environment

Collaboration is more than dividing tasks—it is establishing shared understanding, shared standards, and feedback loops. Across coursework, I practiced:

- **Iterative review and refinement**: working in small increments, using feedback to improve design decisions rather than only polishing final output.
- **Shared quality practices**: applying consistent patterns (clear module boundaries, readable naming, defensive validation) so teammates can extend work safely.
- **Code review behaviors**: explaining intent, calling out trade-offs, and using review feedback to reduce risk (e.g., tightening validation and improving test coverage).

As a concrete example outside the artifact itself, the program reinforced how to treat unit tests and integration tests differently (mocked isolation vs. real dependencies) and how to structure changes so they are reviewable and easy to reason about.

Communicating with stakeholders

The program emphasized that successful software requires technical decisions that are explainable to non-technical audiences. I learned to:

- Translate requirements into clear, testable behaviors (what "done" looks like).
- Explain trade-offs (performance vs. cost, usability vs. strictness, security vs. convenience) in plain language.

- Produce written deliverables that support decisions, such as architecture summaries, enhancement narratives, and review notes.

That same communication mindset is reflected in the ePortfolio's organization: the self-assessment introduces the whole portfolio, and each narrative explicitly states what was learned, what changed, and why it matters.

Data structures and algorithms (applied, not theoretical)

Across the program, I strengthened my ability to choose algorithms and data structures based on user behavior and system constraints. I learned to think in terms of:

- **Workload patterns** (repeated reads vs. frequent writes, predictable queries vs. exploratory search)
- **Complexity trade-offs** (time, space, and maintainability)
- **Operational impact** (latency expectations, cache invalidation, fallbacks)

This ePortfolio demonstrates applied algorithmic thinking through search and performance improvements (autocomplete, fuzzy matching, caching) while keeping the system maintainable and testable.

Software engineering and databases (building production-oriented systems)

The program helped me connect software engineering practices with database realities:

- Separation of concerns (UI, business logic, data access)
- Service boundaries that support reuse and testing

- Database indexing and aggregation for performance and analytics
- Observability and operational readiness (logging, health checks, audit trails)

I also learned that database design is not only schema—it includes query design, index strategy, and how data supports reporting and decision making.

Security mindset

A security mindset means anticipating adversarial and accidental failure modes. The program reinforced that security must be designed in from the beginning through:

- **Least privilege**: restricting actions based on roles and responsibilities
- **Explicit validation**: accepting only known-good inputs and handling errors safely
- **Auditability**: creating traceability for sensitive operations
- **Defense in depth**: layering protections such as rate limiting and security headers

Just as importantly, I learned to treat security as a continuous process: identify risks, reduce attack surface, and add evidence (tests, logs, documentation) that controls exist and work.

How the artifacts fit together

This ePortfolio focuses on a single artifact enhanced across the three CS 499 categories. The materials are organized to tell a coherent story:

- **Artifact overview**: what the original project was and why it was selected
- **Enhancement narratives**: how the artifact was improved in software engineering, algorithms/data structures, and databases—with learning-focused reflection

- **Written informal code review**: a peer/manager-oriented review that explains design issues, planned vs. implemented enhancements, and trade-offs
- **Rubric evidence map**: a quick reference linking course outcomes to concrete evidence in the repository

Together, these artifacts demonstrate my growth from building functional coursework solutions to delivering systems that are structured, secure, performance-aware, and communicate their intent clearly.